

Übungsblatt 1a

Infrastruktur & Organisation

5430UE Web and Data Engineering

15. April 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

Lernziele

- Den **Ablauf** der Lehrveranstaltung und die Prüfungsleistungen kennen
- Das **Projekt** organisatorisch einordnen können (Team, Abgabe, Bewertung)
- Die **Entwicklungsumgebung** (FIM-Account, GitLab, Git) einrichten
- Die Versionsverwaltung mit **Git** sicher beherrschen (Initialisierung, Commits, Branching)

Übungsablauf

- **Vorlesung:** wöchentlich, Übungsblätter werden begleitend ausgegeben
- **Übungsstunde:** Aufgaben gemeinsam besprechen, dann zur Lösungsfolie wechseln
- **Projektarbeit:** parallel zur Vorlesung in Teams von **3 Personen**
- **Bewertung:** Klausur (am Semesterende) + individuell bewertete Projektnote
- **Tools:** Stud.IP für Materialien, **FIM GitLab** für Projektcode, Forum/Chat für Austausch

Projekt — Ablauf, Technologien, Benotung

Ablauf

- Team-Bildung (3 Personen) zu Beginn des Semesters über Stud.IP-Teams
- Phase I (Konzept, erste Implementierung) → Zwischenabgabe
- Phase II (vollständige Umsetzung) → finale Abgabe vor Klausurtermin
- Abgabe ausschließlich über **FIM GitLab** (gitlab.fim.uni-passau.de); FIM-Account erforderlich

Technologien

- HTML, JavaScript, CSS für das Frontend (Spring MVC + Thymeleaf)
- Java mit **Spring Boot**, Spring Data, Spring Security für das Backend
- REST-Schnittstelle, persistente Speicherung
- **GitFlow** mit main / develop / feature/* Branches

Benotung

- **Individuelle** Note pro Teammitglied — Bewertung erfolgt auf Basis der Commit-Historie
- Achten Sie darauf, mit Ihrem **eigenen** GitLab-Account zu committen
- Projektnote kann auf die Klausur angerechnet werden
- Saubere Git-Historie und Tool-Disziplin werden mit bewertet



Modul A — Organisatorisches

Übung 1a.A.1: Kursüberblick

Beantworten Sie folgende Fragen zur Kursorganisation:

1. Welche zwei Prüfungsleistungen gibt es in dieser Lehrveranstaltung?
2. In welchen Teamgrößen wird das Projekt bearbeitet?
3. Wo wird der Projektcode eingecheckt, und warum ist das Tool-Setup bereits zu Beginn wichtig?

Übung 1a.A.1: Kursüberblick — Lösung

1. **Klausur** (schriftliche Prüfung) und **Projektarbeit** (Implementierungsprojekt in Teams).
2. Teams von **3 Personen**.
3. Code wird auf dem **FIM GitLab** eingchecked. Frühes Setup ist wichtig, da individuelle Commits für die Benotung herangezogen werden - fehlende Commit-Historie kann nicht nachträglich ergänzt werden.

Modul B — Infrastruktur & Git

Übung 1a.B.1: FIM-Account & GitLab

Für den Zugang auf das **FIM GitLab** benötigen Sie einen FIM-Account.

1. Besuchen Sie die [FIM-Webseite zum Accountmanagement](#) und beantragen Sie bei Bedarf einen Account.
2. Falls Sie aus dem Universitätsnetz heraus arbeiten müssen, verwenden Sie [OpenVPN](#).
3. Stellen Sie sicher, dass Sie Zugang zum gitlab.fim.uni-passau.de haben.
4. Legen Sie ein eigenes Repository `wde-exercises` an und klonen Sie es lokal.

Übung 1a.B.1: FIM-Account & GitLab — Lösung

Typische Stolpersteine:

- **SSH-Schlüssel:** Hinterlegen Sie einen SSH-Public-Key in Ihrem GitLab-Profil, sonst müssen Sie bei jedem Push das Passwort eingeben.
- **VPN nur bei Bedarf:** Das FIM GitLab ist auch von außerhalb des Uni-Netzes erreichbar — VPN nur, wenn Sie auf interne Dienste zugreifen.
- **Eigener Account beim Commit:** Stellen Sie `git config user.email` auf Ihre FIM-E-Mail, sonst werden Commits später nicht Ihrer Person zugeordnet.

```
git config --global user.name "Max Mustermann"  
git config --global user.email "mustermann@fim.uni-passau.de"
```

Übung 1a.B.2: Einführung Git — Workflow

Führen Sie die ersten Schritte im Repository durch:

1. Wie initialisiert man ein lokales Repository (falls nicht geklont)?
2. Erstellen Sie eine Datei `hello.txt` mit `Hallo WDE!`.
3. Committed und pushen Sie die Datei. Notieren Sie die Befehle.
4. Was ist der Unterschied zwischen `git fetch` und `git pull`?

Übung 1a.B.2: Einführung Git — Lösung

```
# Initialisieren
git init
echo "Hallo WDE!" > hello.txt

# Staging und Commit
git add hello.txt
git commit -m "Add hello.txt"

# Pushen
git push origin main
```

Unterschied: `git fetch` lädt nur neue Commits vom Remote, ohne den lokalen Branch zu verändern. `git pull` macht zusätzlich einen merge in den aktuellen Branch.

Übung 1a.B.3: Git-Grundlagen — Branching

Sie arbeiten an einem Feature `user-login` in Ihrem Projekt.

1. Welchen Git-Befehl verwenden Sie, um einen neuen Branch `feature/user-login` zu erstellen und direkt zu wechseln?
2. Was ist der Unterschied zwischen `git merge` und `git rebase`?
3. Warum empfiehlt das GitFlow-Prinzip, Arbeiten in Feature-Banches auszulagern?

Übung 1a.B.3: Git-Grundlagen — Branching — Lösung

1.

```
git checkout -b feature/user-login  
# oder modern:  
git switch -c feature/user-login
```

2. `git merge` erzeugt einen Merge-Commit und bewahrt die vollständige Branch-Historie. `git rebase` schreibt die Commits auf dem Feature-Branch um, sodass sie linear auf dem Zielbranch aufsetzen.
3. Feature-Branches isolieren Arbeiten voneinander: `main` bleibt jederzeit deploybar, parallele Features stören sich nicht, und Code-Reviews sind vor dem Merge möglich.

Modul C – Zusammenfassung

Wrap-Up: Infrastruktur

- Kursorganisation ist verstanden (Projekt + Klausur)
- FIM-Account ist aktiv und GitLab-Zugang funktioniert
- Basis-Git-Workflows (Commit, Push, Branch) sind bekannt
- Das Tool-Design ermöglicht individuelle Bewertung der Projektleistung